

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงงาน

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการพัฒนาโปรแกรมการจับกลุ่มพีเจอรันนิวโรลเน็ตเวิร์ค ซึ่งเนื้อหาในบทนี้จะกล่าวถึง ทฤษฎีทาง Image processing นิวโรลเน็ตเวิร์คแบบแพร่กลับ (Back-propagation Neural Network) และการแบ่งกลุ่มของข้อมูล (Clustering Algorithm) ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการพัฒนา และประเมินประสิทธิภาพของโปรแกรมการจัดกลุ่มบนนิวโรลเน็ตเวิร์ค

2.1 ระบบการมองเห็นภาพ (Vision System)

กลไกระบบการมองเห็นภาพ (Vision System) นั้น จะหมายถึงความรวมถึงทุกสิ่งที่เป็นที่สามารถแปลงภาพนั้นๆ เป็นรหัสดิจิทัล เพื่อนำไปใช้ในระบบคอมพิวเตอร์ได้, การปรับปรุงเปลี่ยนแปลงข้อมูล และการเสนอภาพที่ได้มาหลังจากการปรับปรุงเปลี่ยนแปลงข้อมูลแล้ว ระบบการมองเห็นภาพนี้ความยุ่งยากจะขึ้นอยู่กับการใช้งาน ซึ่งสามารถแบ่งเป็น 3 ขั้นตอนที่สำคัญ ดังนี้

1. การได้มาซึ่งภาพ (Image Acquisition)
2. กระบวนการการประมวลผลภาพ (Image Processing)
3. ผลที่ได้ หรือ การแสดงผล (Output or Display)

ในปัจจุบันได้มีการประยุกต์ใช้งานระบบการมองเห็นภาพอยู่มากมาย เช่น การใช้บาร์โค้ด, การพิมพ์สิ่งต่างๆ, การประยุกต์ใช้งานในโรงงาน

1. การได้มาซึ่งภาพ (Image Acquisition)

การได้มาซึ่งภาพ หมายถึง การแปลงภาพในลักษณะทางกายภาพ ให้เป็นเซตข้อมูลทางดิจิทัล ซึ่งเซตข้อมูลนี้จะถูกส่งไปยังหน่วยประมวลผลต่อไป ฟังก์ชันของการได้มาซึ่งภาพนี้แบ่งเป็น 4 เฟส คือ

- 1) การส่องสว่าง (Illumination)
- 2) รูปแบบของภาพ หรือ การทำให้ภาพชัดขึ้น (Image formation or Focusing)
- 3) การตรวจจับภาพ หรือ การรับภาพ (Image detection or Sensing)
- 4) รูปแบบผลสัญญาณที่ได้จากกล้อง (Formatting camera output signal)

การส่องสว่างเป็นตัวแปรสำคัญที่มีอิทธิพลต่อสัญญาณอินพุต (Input signal) ที่จะส่งต่อไปให้กับ ระบบการมองเห็นภาพ เพราะฉะนั้นเราจึงต้องออกแบบ ให้มีการส่องสว่างที่เหมาะสมกับ

การใช้งานที่แตกต่างกัน โดยที่ชนิดและวิธีการส่องสว่างของแหล่งกำเนิดแสง จะมีผลต่อกำลังงานของแสงที่ส่งออกมา ซึ่งจะมีผลต่อกระบวนการประมวลผลภาพ และผลของสัญญาณที่ได้รับ

2. กระบวนการประมวลผลภาพ (Image Processing)

กระบวนการประมวลผลภาพ คือ การสร้างภาพใหม่โดยการแยกส่วนของข้อมูลที่เราสนใจ กับ สิ่งรบกวน (Noise) ออกจากกัน โดยการทำงานพื้นฐานของการประมวลผล คือ การกำจัดสิ่งรบกวนของภาพ (Noise elimination) , การปรับแต่งของภาพให้ดีขึ้น (Edge enhancement) , การกรองภาพ (Filtering) , การปรับปรุงเปลี่ยนแปลงค่าระดับเกรย์ (Grey scale modification) โดยทั่วไปแล้วจะใช้ฮาร์ดแวร์ในการประมวลผลภาพ แต่ถ้ามีการประมวลผลภาพที่ซับซ้อนขึ้น ก็จะใช้ทั้ง ฮาร์ดแวร์ และซอฟต์แวร์ ซึ่งการซับซ้อนของการประมวลผลจะขึ้นอยู่กับลักษณะของการใช้งาน และมีวิธีการประมวลผล 3 อย่าง คือ

1) ที่จุดเดียวกันในรูปภาพเดียวกัน (Point by point in one image)

คือ การสร้างภาพใหม่โดยการเปลี่ยนค่าแบบจุดต่อจุด โดยจุดหนึ่งคือจุดในรูปภาพเดิม และอีกจุดคือจุดในรูปภาพใหม่ที่ได้รับการปรับปรุงเปลี่ยนแปลงจากรูปภาพเดิมแล้ว เช่น การแปลงภาพในระบบเลขฐาน 2 จากจุดที่มีค่าเป็น 0 ถูกเปลี่ยนเป็น 1 และจากค่า 1 ถูกเปลี่ยนกลับเป็น 0

2) ที่จุดเดียวกันในรูปภาพที่แตกต่างกัน (Using corresponding points on different images)

คือ การสร้างภาพใหม่โดยการจับคู่แต่ละจุดจากแหล่งภาพ 2 แหล่ง หรือมากกว่า จุดของรูปภาพที่แตกต่างกันสองจุดนี้ จะนำมาลบกันเพื่อหาค่าที่เปลี่ยนแปลงไป ข้อมูลของจุดในภาพใหม่จะเกิดจากการรวมกันของตัวเลขที่แตกต่างกันของภาพ 2 แหล่งนั้น

3) ที่บริเวณจุดนั้นในภาพนั้น (Using regional points in one image)

คือ การสร้างภาพใหม่โดยการหาค่าเฉลี่ยรอบๆจุดนั้นในภาพนั้น ค่าของจุดในภาพใหม่คือ ค่าเฉลี่ยของจุดในภาพนั้นอันเดิม

3. ผลที่ได้และการแสดงผล (Output or Display)

รูปแบบของการแสดงผลนั้นขึ้นอยู่กับว่า จะนำข้อมูลที่ได้ไปใช้ต่อในลักษณะใด หรือแสดงผลอย่างไรจึงจะเหมาะสมที่สุด เช่น การแสดงผลโดยการพิมพ์ , การแสดงผ่านหน้าจอ , การแสดงผลในรูปแบบของสัญญาณควบคุม เป็นต้น

2.1.1 หลักการเบื้องต้นของการประมวลผลภาพ (Image Processing Fundamentals)

1. พิกเซล (Pixel)

ในภาพหนึ่งๆ เราสามารถอธิบายได้ด้วยเมทริกซ์ของจุดพิกเซลขนาด $N \times M$ โดยใช้คู่ลำดับ $p(i, j)$ แทนค่าของจุดแต่ละจุด โดย i และ j ต้องเป็นจำนวนเต็มบวกหรือศูนย์เท่านั้น และ $p(i, j)$ นี้ จะบ่งชี้ความเข้มของแสงที่จุดนั้นๆ ของภาพ

ค่าที่กำกับแต่ละพิกเซลจะแสดงถึงค่าเฉลี่ยของความเข้มแสงในภาพที่จุดพิกเซลนั้นแทนอยู่ โดยที่ค่าของพิกเซลนี้จะเขียนแทนด้วย P_{ij} ซึ่งมีค่าตั้งแต่ 0 ถึง 1

2. ตำแหน่งพิกเซล (Pixel Location)

ดังที่ได้กล่าวมาแล้วว่า ในภาพหนึ่งๆ นั้น เราสามารถแทนอาร์เรย์ (Array) $N \times M$ และค่าในแต่ละจุดพิกเซล จะหมายถึงค่าเฉลี่ยของความเข้มของแสงที่ตกกระทบถึงภาพที่จุดพิกเซลนั้น

2.1.2 ระดับเกรย์ (Grey Scale)

หากเราต้องการค่าของข้อมูลที่ละเอียดมากขึ้น จะต้องเพิ่มจำนวนบิตในการแสดงค่าของแต่ละพิกเซล เช่น ถ้าต้องการแบ่งความเข้มของการส่องสว่างให้มี 4 ระดับ ก็ต้องใช้เลขฐานสอง 2 บิต, 4 บิต สำหรับ 16 ระดับ และ 8 บิต สำหรับ 256 ระดับ ซึ่งจำนวนระดับที่ใช้ในระดับเกรย์นี้ มักเป็นเลขยกกำลังของ 2 ส่วนค่าต่ำสุด คือ 0 ถูกกำหนดให้เป็นสีดำ และ 1 หรือตัวเลขที่น้อยกว่าค่าสูงสุดของระดับเกรย์อยู่ 1 (เช่น 15 สำหรับระดับเกรย์ 16 ระดับ) ถูกกำหนดให้เป็นสีขาว ค่าที่กำหนดไว้ในแต่ละพิกเซลมักเป็นจำนวนเต็ม

Grey Scale

Grey Value Range

2^1	2 values	0, 1
2^3	8 values	0 to 7
2^4	16 values	0 to 15
2^8	256 values	0 to 255

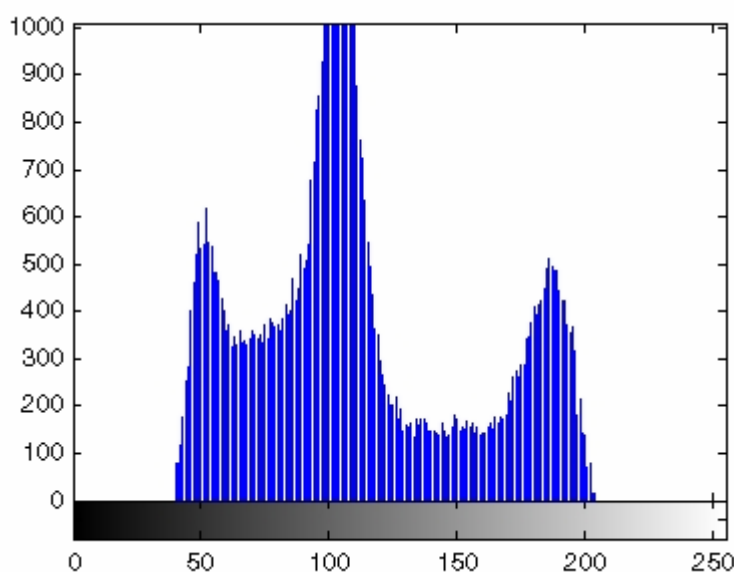
ในยุคแรกๆ ของระบบการมองเห็นภาพ (Vision System) จะใช้ระบบเลขฐานสอง แต่ในปัจจุบัน เทคโนโลยีของไมโครโพรเซสเซอร์ (Microprocessors) เข้ามามีบทบาทมากขึ้น ซึ่งไมโครโพรเซสเซอร์จะมีหน่วยความจำอย่างน้อย 8 บิต เพราะฉะนั้นการแบ่งระดับเป็น 16, 64 หรือ 256 จึงเป็นเรื่องธรรมดา และในตอนนี้กำลังจะมีมากกว่า แต่ในการมองเห็นของมนุษย์สามารถแยกความแตกต่างได้เพียง 10 ถึง 15 ระดับ เท่านั้น ดังนั้นการแบ่งโดยละเอียดเป็น 64 หรือ 256 ระดับ ทำให้มนุษย์ไม่สามารถแยกความแตกต่างได้ จึงนำไปประยุกต์ใช้กับงานการประมวลผลภาพ หรือ ใช้ในกระบวนการทางอุตสาหกรรมต่างๆ

จึงอาจกล่าวได้ว่าจำนวนระดับเกรย์เป็นตัวจำกัดรายละเอียดของภาพ ยิ่งแบ่งระดับเกรย์เป็นหลายระดับ ก็ยิ่งเพิ่มคุณภาพของภาพนั้นด้วย และการเพิ่มจำนวนพิกเซล เช่น จาก 32×32 เป็น 250×250 ก็จะเป็นการเพิ่มรายละเอียด (Resolution) และรายละเอียด (Detail) ของภาพซึ่งจะแตกต่างกับการขยายภาพ (Zoom) เพราะการขยายภาพ เป็นการเพิ่มขยายของแต่ละพิกเซลให้ใหญ่ขึ้น ไม่ได้เป็นการเพิ่มจำนวนพิกเซล แต่การแบ่งระดับเกรย์เป็นการเพิ่มจำนวนของพิกเซล

2.1.3 ฮิสโตแกรม (Histogram)

ฮิสโตแกรมเป็นการแสดงให้เห็นถึงสถิติของการนับจำนวนพิกเซล ที่มีค่าความเข้มแต่ละค่าหนึ่งในภาพระดับเกรย์ จากรูป 2.3 แกน x ในกราฟแสดงค่าระดับเกรย์ และแกน y แสดงค่าจำนวนพิกเซลในระดับเกรย์นั้น โดยฮิสโตแกรมมีขั้นตอนการสร้างดังนี้

- 1) ทำการดิจิไทซ์ (Digitizing) ภาพ
- 2) นับจุดพิกเซลในแต่ละระดับเกรย์
- 3) พล็อตกราฟระหว่างจำนวนจุดกับระดับเกรย์



รูปที่ 2.1 แสดงฮิสโตแกรมของ 256 ระดับเกรย์

รูปร่างของฮิสโตแกรมสามารถบอกลักษณะบางประการของภาพได้ เช่น ถ้าฮิสโตแกรมมีลักษณะที่แคบ จะหมายถึงขาดการคอนทราสต์ (Contrast, การแยกแยะความผิดแผกของสี) ในภาพนั้น

ฮิสโตแกรมมีประโยชน์ในการกำหนดระดับเทรชโฮลด์ (Threshold) ซึ่งก็คือการเปลี่ยนภาพระดับเกรย์ให้เป็นภาพระบบเลขฐานสอง (Binary) หรือเพื่อการปรับปรุงบางส่วนของสเปกตรัม (Spectrum) ของระดับเกรย์

2.1.4 การแปลงระดับเกรย์ (Grey-level Transformation)

1) การเปลี่ยนแปลงความสว่าง(Global Alternative in brightness)

เป็นการใช้ค่าคงที่บวกหรือลบออกจากทุกพิกเซลของภาพ เพื่อเพิ่มหรือลดความสว่างของภาพ

2) การทำเทรชโฮล (Threshold)

เป็นการเปลี่ยนแปลงหรือหาแนวโน้มของค่าระดับเกรย์ในภาพเพื่อทำให้เป็นดิสครีท (Discrete) มากขึ้น โดยจะนำภาพมาทำฮิสโตแกรม แล้วกำหนดค่าระดับเกรย์ที่แน่นอนขึ้นมา เพื่อที่จะเป็นจุดตัดทำเทรชโฮล จากนั้นจะทำการตัดหรือปิดส่วนของระดับเกรย์ที่เราไม่ต้องการออกไป

ประโยชน์ของเทรชโฮล ได้แก่ การทำให้เป็นภาพไบนารี หากดูจากรูปที่ จุดที่อยู่ในช่วงทางซ้ายมือทั้งหมดของจุดตัดเทรชโฮล T จะถูกทำเป็นสีขาว และจุดในช่วงทางขวามือทั้งหมดจะถูกทำเป็นสีดำ หรือ การช่วยให้หาขอบของภาพได้ง่ายขึ้น เป็นต้น แต่ทั้งนี้ก็เป็นการยากที่จะกำหนดจุดตัดเทรชโฮลที่ดีที่สุดออกมาได้

3) บันช์ซิง (Bunching) และการควอนไทซ์ (Quantize)

ในบางครั้ง การบันช์ซิง จะหมายถึง การควอนไทซ์ ด้วย หรือ ใช้ลดระดับเกรย์ของภาพที่ไม่ต้องการลง

หมายเหตุ การควอนไทซ์เป็นการประมาณค่าระดับเกรย์ให้เป็นจำนวนเต็ม ซึ่งจะเป็นไปตามกฎ คือ ถ้าเป็นจุดทศนิยมให้ปัดขึ้นทั้งหมด และการปัดนี้ก็คือความผิดพลาดที่เกิดขึ้น

4) สปลิตติง (Splitting)

เป็นการเพิ่มความแตกต่างระหว่าง 2 กลุ่มของระดับเกรย์ เช่น ถ้าเรามีตัวอักษรเขียนอยู่บนฉาก โดยตัวเลขมีระดับเกรย์ที่ 98 , ฉากมีระดับเกรย์ที่ 99 ซึ่งตาของมนุษย์ไม่สามารถแยกแยะความแตกต่างเพียงเท่านี้ได้ และจะได้ฮิสโตแกรมที่มีลักษณะแบน เพราะฉะนั้นจึงแก้ปัญหาด้วยการสปลิตฮิสโตแกรม โดยทำการดึงค่า 99 ขึ้นเป็น 120 และดึงค่า 98 เป็น 80 ซึ่งจะทำให้ระหว่างตัวเลขและตัวอักษรมีความแตกต่างกันมากขึ้นจนสามารถสังเกตได้

เทคนิคนี้จะมีประโยชน์มาก ในกรณีที่เราต้องดึงเอาเฉพาะบางส่วนของภาพออกมา

2.1.5 พื้นฐานและระบบของสีโมเดล RGB (Red, Green, Blue)

ในโมเดลนี้ สีแต่ละสีจะอยู่ในรูปของสีปฐมภูมิ (แดง, เขียว, น้ำเงิน) โมเดลนี้มีโครงสร้างอยู่ในระบบของพิกัดคาร์ทีเซียน (Cartesian coordinate) มีลักษณะเป็นทรงลูกบาศก์ ดังรูปที่ ค่าสีแดง, เขียว, น้ำเงิน จะอยู่ที่มุมทั้งสามในลักษณะเดียวกัน ส่วนสีดำจะอยู่ที่จุดกำเนิด สีขาวอยู่ที่มุมที่มีระยะทางไกลที่สุดจากแหล่งกำเนิด ค่าของระดับสีเทา (Grey Scale) จะอยู่บนเส้นที่เชื่อมระหว่างสีดำกับสีขาว ค่าสี คือ จุดที่อยู่บนผิวหรือในลูกบาศก์ถูกกำหนดค่าโดยเวกเตอร์ที่ชี้ออก

จากจุดกำเนิด เพื่อความสะดวกเราจะสมมุติให้ค่าสีถูกนอร์มัลไลซ์ (Normalize) ทั้งสามสี โดยให้ มีค่าอยู่ในช่วง 0 ถึง 1 ลูกบาศก์ที่แสดงในรูปที่ 2.1.5 นี้ จึงเป็นลูกบาศก์หนึ่งหน่วย

ภาพในโมเดล RGB ประกอบด้วยภาพสามระนาบที่เป็นอิสระจากกัน สำหรับแต่ละสี ปริมาณเมื่อป้อนเข้าไปในมอนิเตอร์ (monitor) ที่เป็นแบบ RGB ภาพทั้งสามสีจะรวมตัวกันที่จอภาพกลายเป็นภาพสีผสม ดังนั้นการใช้โมเดล RGB ในการประมวลผลภาพนั้นจะสมเหตุสมผล เมื่อภาพถูกแยกออกโดยธรรมชาติให้อยู่ในเทอมของทั้งสามสี กล้องภาพสีส่วนใหญ่ที่ให้ภาพสีดิจิทัลจะอยู่ในรูปแบบของโมเดล RGB ดังนั้นโมเดลนี้จึงเป็นโมเดลที่สำคัญมากในการประมวลผลภาพ

2.1.6 ความสว่าง (Brightness)

ความสว่างเป็นคุณสมบัติของแสงที่มนุษย์เป็นผู้กำหนดให้ระหว่างความสว่างมากที่สุดกับความมืด ความสว่างเป็นสิ่งที่รับรู้ได้แต่ไม่สามารถสัมผัสได้ ความสว่างสามารถรับรู้ได้เมื่อมีแสงตกกระทบกับเรตินารูปแท่งและรูปกรวยในตาของคนเรา ซึ่งจะก่อให้เกิดการตอบสนองที่ไม่เป็นเชิงเส้นและสลับซับซ้อน ความไวในการรับแสงของคนเราจะลดลงเมื่อแสงมีขนาดใหญ่มากขึ้น สำหรับความสว่างที่อยู่ในรูปภาพนั้น สามารถแสดงได้โดย Brightness Histogram ซึ่งเป็นกราฟการกระจายตัวของกลุ่มตัวเลข ในกราฟนี้จะแสดงถึงระดับสีเทาของแต่ละพิกเซลที่อยู่ในรูปภาพ หรืออาจจะกล่าวได้ว่าเป็นการแสดงให้เห็นถึงจำนวนของพิกเซลในรูปภาพที่อยู่ในแต่ละระดับของสีเทามีอยู่เป็นจำนวนเท่าไร

2.1.7 ชนิดของรูปภาพในกล่องเครื่องมือของโปรแกรมเมทแลป

เครื่องมือในการประมวลผลรูปภาพรองรับรูปแบบพื้นฐาน 4 อย่างของรูปภาพ คือ

- ดัชนีรูปภาพ (Index images)
- ความหนาแน่นของรูปภาพ (Intensity images)
- ไบนารีของรูปภาพ (Binary images)
- อาร์จีบีของรูปภาพ (RGB images)

1. ดัชนีรูปภาพ (Index images)

ดัชนีรูปภาพประกอบไปด้วยเมทริกซ์ของข้อมูล X และเมทริกซ์ของสีที่จะนำไปเทียบเคียงเมทริกซ์ของข้อมูลสามารถแบ่งออกเป็นคลาสของ uint8, uint16 หรือ double ส่วนเมทริกซ์ของสีนั้นจะเป็นอาร์เรย์ขนาด $m \times 3$ ของคลาส double ที่ประกอบด้วยเลขทศนิยมที่อยู่ในช่วง $[0,1]$ แต่ละแถวของการเทียบเคียงนั้นจะระบุคอมโพเนนต์ สีแดง, สีเขียว และสีน้ำเงินของแต่ละสีนั้น ดัชนีรูปภาพจะทำการเทียบเคียงโดยตรงของแต่ละค่าในพิกเซลนั้นไปยังค่าเทียบเคียงสี สีในแต่ละพิกเซลบนรูปภาพนั้นจะถูกกำหนดโดยใช้ค่าที่มีลักษณะเช่นเดียวกันกับ X เป็นดัชนีในการ

เทียบเคียง ค่าของจุดแรกแทนแถวแรกในการเทียบเคียง ค่าของจุดที่สองแทนแถวที่สอง และเป็น
เช่นนี้เรื่อยไป

การเทียบเคียงสีมักจะถูกเก็บด้วยดัชนีรูปภาพและจะนำมาใช้อย่างอัตโนมัติกับรูปภาพเมื่อเรียกใช้ฟังก์ชัน imread อย่างไรก็ตามในการใช้งานนั้น ไม่ได้

ถูกจำกัดการใช้งานเฉพาะค่าพื้นฐานของการเทียบเคียงสีเท่านั้น แต่ยังสามารถเรียกใช้งานค่าใดๆในการเทียบเคียงสีได้ รูปภาพดังต่อไปนี้แสดงถึงโครงสร้างของดัชนีรูปภาพ พิกเซลที่อยู่ในรูปภาพจะถูกแสดงด้วยเลขจำนวนเต็ม ซึ่งจะชี้ไปยังค่าของสีที่เก็บอยู่ในการเทียบเคียงสีนั้น

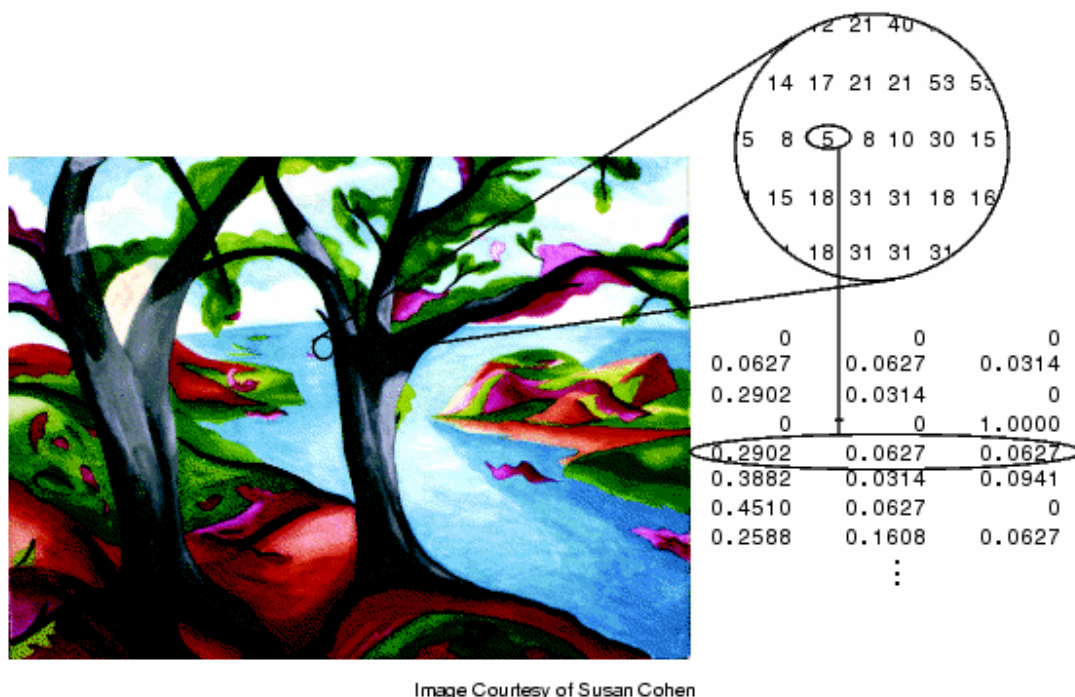


Image Courtesy of Susan Cohen

รูปที่ 2.2 ค่าของฟังก์ชัน f ไปยังค่าเทียบเคียง s ในดัชนีรูปภาพ

- **คลาสและค่าออฟเซตของการเทียบเคียงสี่**

ความสัมพันธ์ระหว่างค่าที่อยู่ในเมทริกซ์รูปภาพกับค่าในการเทียบเคียงสีนั้นขึ้นอยู่กับคลาสของเมทริกซ์รูปภาพ ถ้าเมทริกซ์รูปภาพเป็นคลาสของ double ค่าของจุดแรกจะชี้ไปยังแถวแรกของการเทียบเคียง ค่าของจุดที่สองจะชี้ไปยังแถวที่สอง และเป็นเช่นนี้ไปเรื่อยไป ถ้าเมทริกซ์รูปภาพเป็นคลาสของ uint8 หรือ uint16 มันจะเป็นค่าออฟเซต ค่าของจุด 0 จะชี้ไปยังแถวแรกของการเทียบเคียงสี ค่าของจุดที่หนึ่งจะชี้ไปยังแถวที่สอง และเป็นเช่นนี้เรื่อยไป

ค่าออฟเซตมักจะถูกใช้ในไฟล์ที่มีรูปแบบกราฟิกเพื่อจะทำให้จำนวนของสีที่สามารถรองรับได้มีมากที่สุด จากภาพข้างบนนี้เมทริกซ์รูปภาพเป็นคลาส double เพราะมันไม่มีออฟเซต ดังนั้นค่าของจุดที่อาจจะขึ้นไปยังแถวที่ห้าของการเทียบเคียง

- ข้อจำกัดของการใช้ uint16

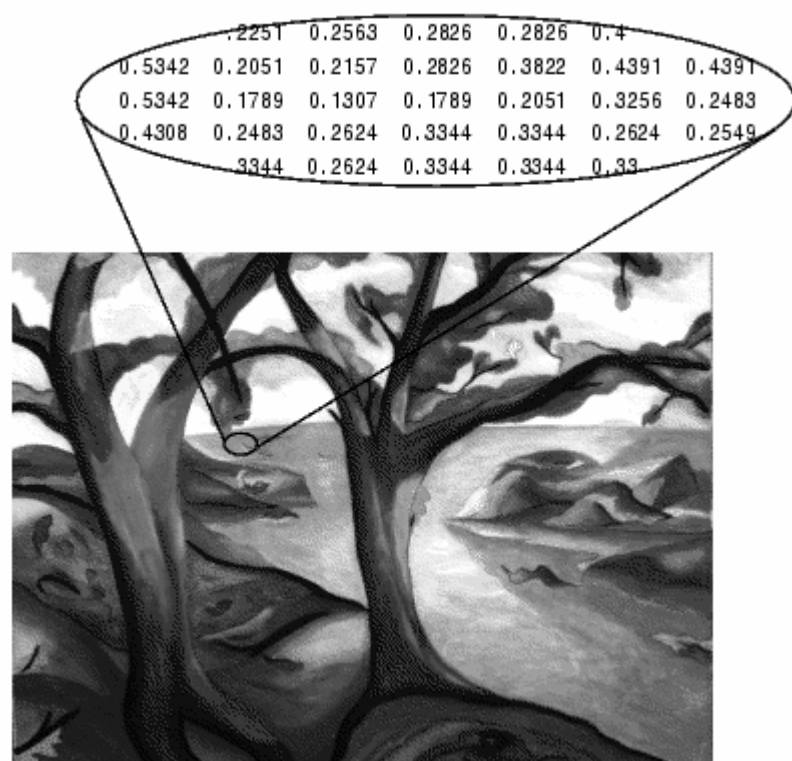
ในกล่องเครื่องมือนี้จะมีข้อจำกัดของการใช้ดัชนีรูปภาพที่เป็น คลาส uint16 เราสามารถที่จะอ่านรูปภาพนี้เข้าไปยังโปรแกรมเมทแลป และแสดงมันได้ แต่ก่อนที่จะทำกระบวนการของดัชนีรูปภาพแบบ uint16 จะต้องทำการเปลี่ยนให้เป็น double หรือ uint8 เสียก่อน ในการที่จะเปลี่ยนเป็น double ให้เรียกใช้ im2double เพื่อที่จะลดรูปภาพไปเป็น 256 สีน้อยกว่านั้น (uint8) เรียก imapprox

2. ความหนาแน่นของรูปภาพ (Intensity images)

ความหนาแน่นของรูปภาพคือเมทริกซ์ข้อมูล I ซึ่งค่าของมันจะแทนความหนาแน่นภายในระยะเวลาหนึ่งๆ โปรแกรมเมทแลปจะเก็บค่าความหนาแน่นของรูปภาพเป็นเมทริกซ์เดี่ยว โดยที่แต่ละค่าในเมทริกซ์ นั้นจะมีลักษณะเช่นเดียวกับหนึ่งพิกเซลในรูปภาพ เมทริกซ์สามารถเป็นคลาสของ double, uint8, uint16

ค่าที่อยู่ในเมทริกซ์ความหนาแน่นจะแทนความหนาแน่นที่หลากหลาย หรือระดับสีเทา

(Grey levels) โดยค่าความหนาแน่น 0 มักจะแทนด้วยสีดำ และค่าความหนาแน่น 1, 255 หรือ 65535 มักแทนด้วยสีขาว



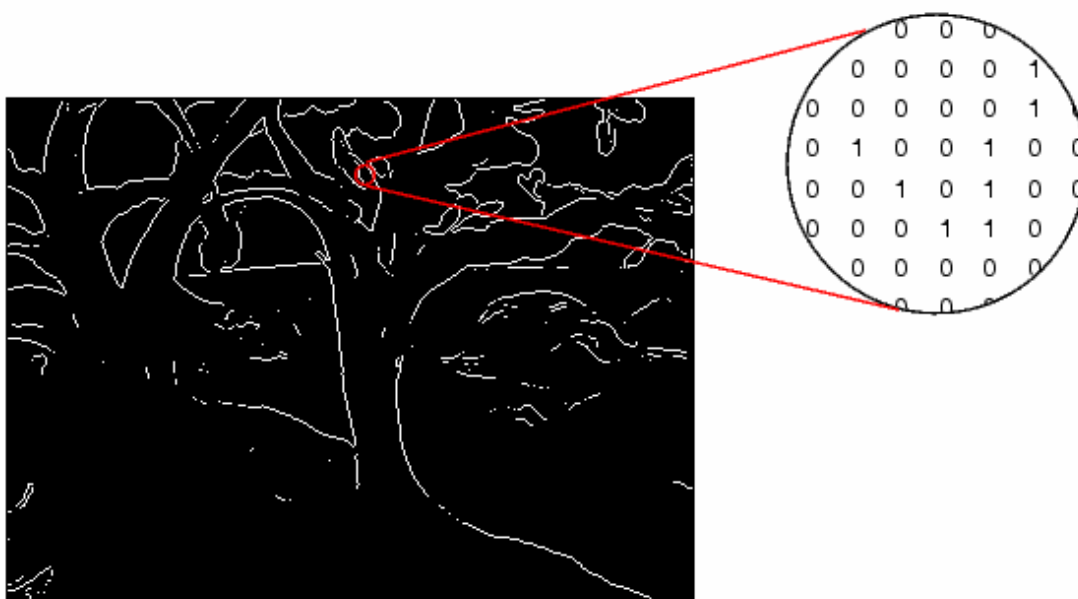
รูปที่ 2.3 ค่าพิกเซลในความหนาแน่นของรูปภาพกำหนดระดับสีเทา

3. ไบนารีของรูปภาพ (Binary images)

ในไบนารีของรูปภาพ แต่ละพิกเซลจะสมมติให้เป็นหนึ่งในสองค่าที่ไม่ต่อเนื่อง ค่าสองค่านี้จะเป็นลักษณะของ เปิด หรือ ปิด ไบนารีของรูปภาพจะถูกเก็บในเมทริกซ์สองมิติของ 0 (ปิดพิกเซล) และ 1 (เปิดพิกเซล)

ไบนารีของรูปภาพถูกพิจารณาเป็นชนิดพิเศษของความหนาแน่นของรูปภาพ ซึ่งประกอบด้วยสีขาวและสีดำเท่านั้น อย่างไรก็ตาม เราสามารถคิดถึงไบนารีของรูปภาพเป็นลักษณะของดัชนีรูปภาพที่มีเพียงสองสีเท่านั้นก็ได้

ไบนารีของรูปภาพจะถูกเก็บเป็นอาร์เรย์ของคลาส double หรือ uint8 (ในกล่องเครื่องมือไม่รองรับไบนารีของรูปภาพที่เป็นคลาส uint16) อาร์เรย์ของคลาส uint8 โดยทั่วไปแล้วจะถูกนำมาใช้มากกว่าอาร์เรย์ของคลาส double เพราะ uint8 ใช้หน่วยความจำน้อยกว่ามาก ในกล่องเครื่องมือการประมวลผลรูปภาพ ฟังก์ชันใดๆที่คืนค่ากลับมาเป็นไบนารีของรูปภาพจะคืนค่ากลับมาเป็นอาร์เรย์ uint8 ถ้าแฟล็กเปิดใช้งานอยู่ ระยะของข้อมูลจะเป็น [0,1] ถ้าแฟล็กปิดใช้งาน ระยะข้อมูลเป็น [0,255]

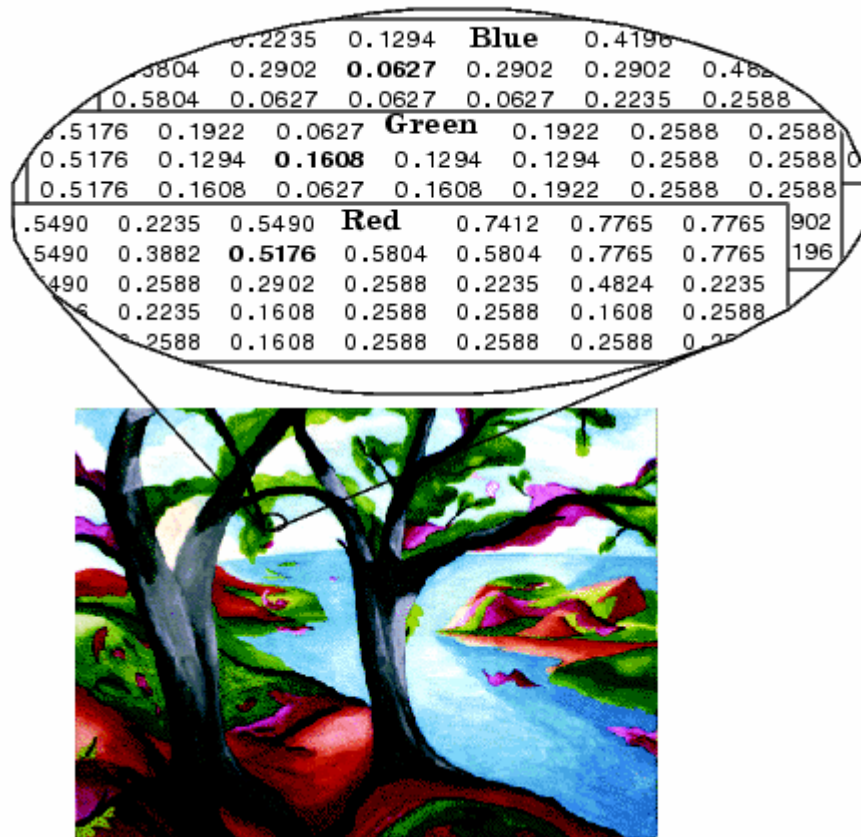


รูปที่ 2.4 พิกเซลในไบนารีของรูปภาพมีค่าที่เป็นไปได้สองค่า คือ 0 หรือ 1

4. อาร์จีบีของรูปภาพ (RGB images)

อาร์จีบีของรูปภาพ หรืออาจเรียกเป็น สีแท้จริงของรูปภาพ (true color image) จะถูกเก็บในโปรแกรมเมทแพลเป็นอาร์เรย์ $m \times n \times 3$ ซึ่งจะกำหนดคอมโพเนนต์สีแดง, สีเขียวและสีน้ำเงินสำหรับแต่ละพิกเซล อาร์จีบีของรูปภาพไม่ได้ใช้แพทเลตของสี สีในแต่ละพิกเซลจะถูกกำหนดโดยการรวมกันของความหนาแน่นของสีแดง, สีเขียว และสีน้ำเงินที่เก็บอยู่ในแต่ละระนาบของสีที่ตำแหน่งของแต่ละพิกเซล รูปแบบไฟล์กราฟิกจะเก็บอาร์จีบีของรูปภาพเป็น 24 บิต ซึ่งแต่ละ

คอมโพเนนต์สีแดง, สีเขียว และสีน้ำเงินจะมีขนาด 8 บิต ซึ่งจะทำให้เกิดจำนวนของสีที่เป็นไปได้ 16 ล้านสี เนื่องจากความละเอียดนี้เอง เราจึงสามารถแทนรูปภาพต่างๆ ที่พบเห็นได้ในชีวิตจริงด้วยรูปภาพแบบสีแท้จริง



รูปที่ 2.5 ระบายสีของรูปภาพอาร์จีบี

ในการกำหนดสีของพิกเซลที่จุด (2,3) จะต้องดูที่อาร์จีบีที่เก็บใน (2,3,1:3) สมมติว่า (2,3,1) ประกอบไปด้วยค่า 0.5176 (2,3,2) ประกอบไปด้วยค่า 0.1608 และ (2,3,3) ประกอบไปด้วยค่า 0.0627 ดังนั้นสีของพิกเซลที่จุด (2,3) คือ

0.5176 0.1608 0.0627

เพื่อที่จะแสดงหลักการของระบายสีสามสีในอาร์จีบีของรูปภาพนั้น โค้ดที่อยู่ข้างล่างนี้เป็น ตัวอย่างของการสร้างอาร์จีบีของรูปภาพที่ประกอบด้วยการไม่รบกวนกันของพื้นที่ในสีแดง, สีเขียว และสีน้ำเงิน จากนั้นจึงสร้างรูปภาพสำหรับแต่ละระบายของสี (แดง,เขียวและน้ำเงิน) โดยจะแสดงแต่ละระบายของสีที่แยกจากกัน และยังคงแสดงถึงรูปภาพต้นฉบับด้วย

```
RGB = reshape(ones(64,1) × reshape(jet(64),1,192),[64,64,3]);
```

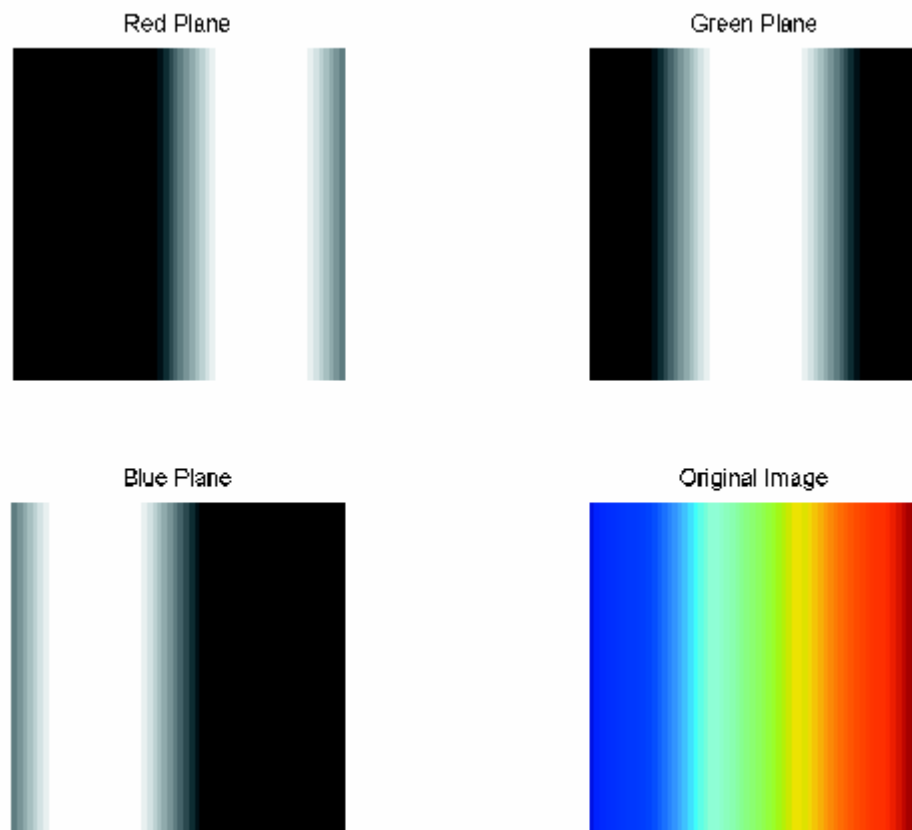
```
R = RGB(:,:,1);
```

```
G = RGB(:,:,2);
```

```

B = RGB(:,:,3);
imshow(R)
figure, imshow(G)
figure, imshow(B)
figure, imshow(RGB)

```



รูปที่ 2.6 ระนาบสีที่แยกจากกันของรูปภาพอาร์จีบี

ในแต่ละสีระนาบสีที่แยกจากกันในรูปแบบประกอบไปด้วยพื้นที่สีขาว สีขาวนี้จะแสดงถึงค่าสูงสุดของแต่ละสีที่แยกจากกัน ตัวอย่างเช่น ในรูปภาพที่เป็นระนาบของสีแดง สีขาวจะแสดงถึงค่าสูงสุดของสีแดง ในขณะที่สีแดงถูกผสมด้วยสีเขียวหรือสีน้ำเงิน พิกเซลที่เป็นสีเทาจะปรากฏออกมา ส่วนพื้นที่ที่เป็นสีดำในรูปภาพนั้นจะแสดงค่าของพิกเซลที่ไม่มีค่าของสีแดงอยู่เลย เช่น $R = 0$

2.1.8 รูปภาพที่เป็นมัลติเฟรมอาร์เรย์ (Multiframe image Arrays)

ในโปรแกรมประยุกต์บางตัวนั้น จำเป็นต้องทำงานร่วมกับรูปภาพจำนวนมากที่เกี่ยวข้องกับเวลาของการเปิดดู เช่น Magnetic Resonance Imaging (MRI) หรือ เฟรมของภาพยนตร์ เป็นต้น

ในกล่องเครื่องมือการประมวลผลภาพนั้นรองรับการเก็บรูปภาพจำนวนมากในอาร์เรย์เดียวกัน แต่ละรูปภาพที่แยกจากกันจะเรียกว่าเฟรม อาร์เรย์รองรับหลายๆเฟรม มันจะถูกนำมาต่อกันเป็นขนาด 4 มิติ เช่น อาร์เรย์ของภาพห้าภาพที่มีอาร์เรย์แบบ 400×300 จะเป็น $400 \times 300 \times 1 \times 5$

โดยการใช้คำสั่ง `cat` ในการเก็บรูปแต่ละรูปที่แยกจากกันในไฟล์ของมัลติเฟรม เช่น ถ้ามีกลุ่มของรูปภาพ A1,A2,A3,A4 และ A5 เราสามารถที่จะเก็บทั้งหมดนี้ภายในอาร์เรย์เดียวโดยใช้คำสั่ง

```
A = cat( 4, A1, A2, A3, A4, A5 )
```

เมื่อต้องการที่จะแยกแต่ละเฟรมออกจากรูปภาพที่เป็นมัลติเฟรม ตัวอย่างเช่น มีรูปภาพที่เป็นมัลติเฟรม `MULTI` คำสั่งต่อไปนี้นี้จะทำการแยกเฟรมที่สามออกมา

```
FRM3 = MULTI( :, :, 3 )
```

ในรูปภาพที่เป็นมัลติเฟรมอาร์เรย์นั้น แต่ละรูปต้องมีขนาดเท่ากันมีจำนวนของระนาบที่เท่ากันด้วย ในดัชนีของรูปภาพที่เป็นมัลติเฟรม แต่ละรูปจะต้องใช้การเทียบเคียงแบบเดียวกัน

ข้อจำกัดในการใช้มัลติเฟรม

ฟังก์ชันจำนวนมากในกล่องเครื่องมือนี้จะทำงานเฉพาะสองหรือสามมิติแรกเท่านั้น เราสามารถใช้อาร์เรย์ขนาดสี่มิติด้วยฟังก์ชันต่อไปนี้ได้ แต่จำเป็นต้องทำงานแต่ละเฟรมโดยแยกจากกัน เช่น ตัวอย่างนี้เป็นการแสดงเฟรมที่เจ็ดในอาร์เรย์ `MULTI`

```
imshow( MULTI( :, :, 7 ) )
```

ถ้าเราส่งอาร์เรย์ไปยังฟังก์ชันและอาร์เรย์นั้น มีมิติมากกว่าการทำงานที่ฟังก์ชันกำหนดไว้ ผลลัพธ์ที่เกิดขึ้นไม่อาจคาดเดาได้ ในบางกรณีฟังก์ชันจะทำงานในเฟรมแรกของอาร์เรย์อย่างถูกต้อง แต่ในกรณีอื่นๆผลลัพธ์ที่เกิดจากการทำงานนั้นอาจไม่มีความหมายอะไร

2.1.9 สรุปชนิดของรูปภาพและคลาสต่างๆ

ตารางนี้เป็นการสรุปวิธีที่โปรแกรมเมทแพลแปลงข้อมูลเมทริกซ์เป็นสีของพิกเซล ซึ่งขึ้นอยู่กับชนิดของรูปภาพและคลาสที่ใช้ในการเก็บ

ตารางที่ 2.1 ความสัมพันธ์ของโปรแกรมเมทแพลกับชนิดของไฟล์ภาพ

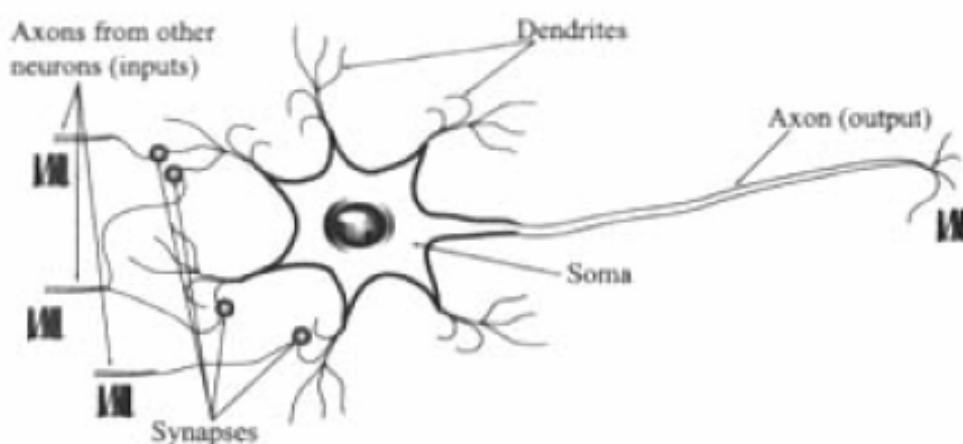
ชนิดรูปภาพ	คลาส double	คลาส uint8 หรือ uint16
ไบนารี	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของหนึ่งกับศูนย์ ซึ่งแฟลทจะเปิดใช้งาน	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของหนึ่งกับศูนย์ ซึ่งแฟลทจะเปิดใช้งานกล่องเครื่องมือไม่รองรับรูปภาพไบนารีที่เป็น uint16
ดัชนี	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของเลขจำนวนเต็มที่อยู่ในช่วง $[1,p]$ การเทียบเคียงสีเป็นอาร์เรย์ขนาด $p \times 3$ ของเลขทศนิยมซึ่งค่าอยู่ในช่วง $[0,1]$	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของเลขจำนวนเต็มที่อยู่ในช่วง $[0,p-1]$ การเทียบเคียงสีเป็นอาร์เรย์ขนาด $p \times 3$ ของเลขทศนิยมซึ่งค่าอยู่ในช่วง $[0,1]$
ความหนาแน่น	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของเลขทศนิยมซึ่งค่าเป็นความสัมพันธ์แบบเชิงเส้นโดยทั่วไปค่าอยู่ในช่วง $[0,1]$ การเทียบเคียงสีเป็นขนาดอาร์เรย์ขนาด $p \times 3$ ของเลขทศนิยม ซึ่งค่าอยู่ในช่วง $[0,1]$ และเป็นระดับของสีเทา	รูปภาพเป็นอาร์เรย์ขนาด $m \times n$ ของเลขจำนวนเต็มซึ่งค่าเป็นความสัมพันธ์แบบเชิงเส้นโดยทั่วไปค่าจะอยู่ในช่วง $[0,255]$ หรือ $[0,65535]$ การเทียบเคียงสีเป็นอาร์เรย์ขนาด $p \times 3$ ของเลขทศนิยม ซึ่งค่าอยู่ในช่วง $[0,1]$ และเป็นระดับของสีเทา
อาร์จีบี	รูปภาพเป็นอาร์เรย์ขนาด $m \times n \times 3$ ของเลขทศนิยม ซึ่งค่าอยู่ในช่วง	รูปภาพเป็นอาร์เรย์ขนาด $m \times n \times 3$ ของเลขจำนวนเต็ม ซึ่งค่าอยู่ในช่วง
ชนิดรูปภาพ	คลาส double	คลาส uint8 หรือ uint16
	$[0,1]$	$[0,255]$ หรือ $[0,65535]$

2.2 นิวรอนเน็ตเวิร์ก (Neural Network : NN)

Neural Network (NN) เป็นคำย่อมาจาก Artificial Neural Network (ANN) ซึ่งมีความหมายว่า “ระบบการทำงานเลียนแบบเซลล์สมองของมนุษย์” โดยที่กล่าวถึงนี้ สามารถใช้เป็นที่ทั้ง โปรแกรมคอมพิวเตอร์ วงจรอิเล็กทรอนิกส์ วงจรดิจิทัล หรือ ระบบผสมทั้งซอฟต์แวร์และฮาร์ดแวร์ แต่ NN นั้นจะต้องมีคุณสมบัติที่สำคัญๆ 2 ข้อ ได้แก่

1. สามารถเรียนรู้ และเก็บความรู้ที่เรียนรู้ไว้ได้ (Learn and Knowledge)
2. สามารถนำความรู้ที่เก็บไว้มาใช้งานในภายหลังได้อย่างถูกต้อง

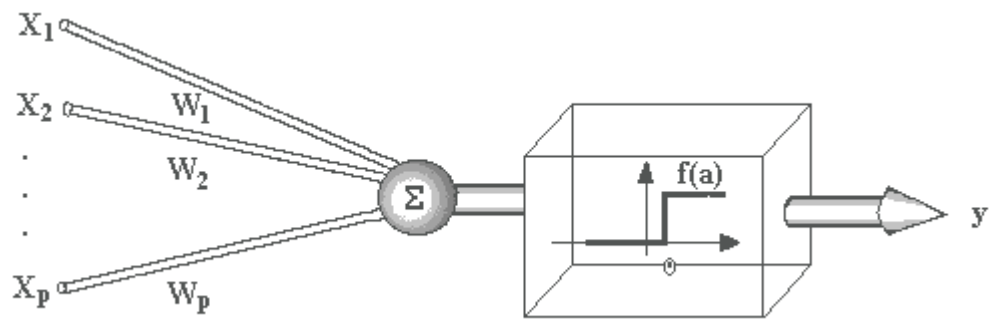
โครงสร้างของสมองมนุษย์ ประกอบด้วยประกอบขึ้นมาจากการรวมกันของเซลล์สมอง (Neuron) โดยจุดที่เชื่อมต่อกันของเซลล์สมองคือ ซิแนปส์ (Synapses)



รูปที่ 2.7 เซลล์สมองของมนุษย์

โดยในเซลล์สมองแต่ละเซลล์จะมีส่วนประกอบที่สำคัญๆ คือ นิวเคลียส (Nucleus) ทำหน้าที่ประมวลผลการกระตุ้นทางไฟฟ้า (Electrical Impulses) ที่ส่งมาจากเซลล์อื่นผ่านทางเดนไดรต์ (Dendrites) และเมื่อประมวลผลแล้วก็จะส่งออกไปสู่แอกซอน (Axon) ไปยังเดนไดรต์ (Dendrites) ของเซลล์อื่นต่อไปเรื่อยๆ และระหว่างซิแนปส์จะมีช่องว่างซิแนปส์ (Synaptic Gap) เพื่อปรับระดับสัญญาณไฟฟ้า

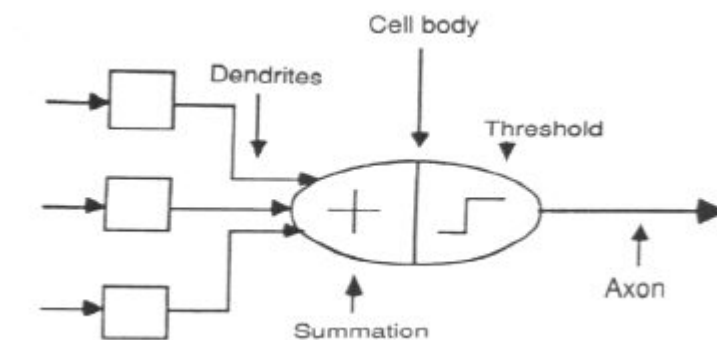
จากโครงสร้างของเซลล์ประสาทดังกล่าวจึงเริ่มมีแบบจำลองเซลล์สมองออกมามากมาย แต่ที่ได้รับความนิยมมากคือแบบจำลองของ Warren McCulloch กับ Walter Pitts (McCulloch & Pitts) ที่นำเสนอด้วยวงจรอิเล็กทรอนิกส์



รูปที่ 2.8 Neural Network ของ McCulloch & Pitts

2.2.1 แบบจำลองทางคณิตศาสตร์ของเซลล์ประสาทเดียว

โครงสร้างแบบจำลองอย่างง่ายของเซลล์ประสาทสามารถเขียนได้ดังแบบนี้



รูปที่ 2.9 โครงสร้างแบบจำลองอย่างง่ายของเซลล์ประสาท

ถ้าสัญญาณที่ส่งออกมาจากแผ่นภาพ คือ n จะสามารถเขียนสมการคณิตศาสตร์ของ n ที่เวลา $t+1$ ได้ดังนี้

$$n_i(t+1) = f\left(\sum_j \omega_{ij} n_j(t) - T_i\right) \quad (2.1)$$

เมื่อ ω_{ij} คือ ค่าความต้านทานของไซแนปส์ที่ j ของเซลล์ประสาทตัวที่ i

$n_j(t)$ คือ สัญญาณที่เซลล์ประสาทตัวที่ j ส่งออกมาที่เวลา t

T_i คือ ค่าเปรียบเทียบกับ ซึ่งเรียกว่า Threshold

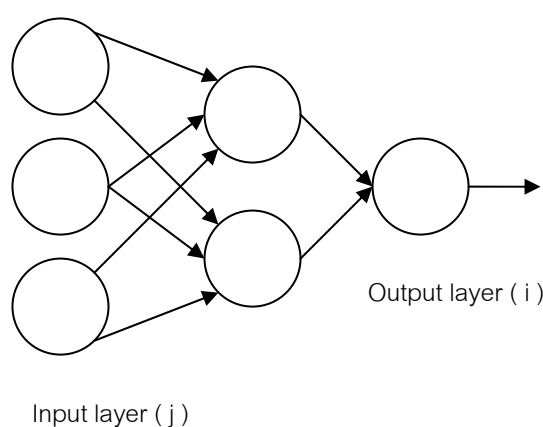
* $f()$ คือ ฟังก์ชัน f ที่ถูกนิยามดังต่อไปนี้

$$f(x) = \begin{cases} 1 & \text{ถ้า } x \geq 0 \\ 0 & \text{ถ้า } x < 0 \end{cases}$$

2.2.2 เทคนิคการเรียนรู้แบบแพร่กลับ (Back propagation)

เทคนิคการเรียนรู้ของโครงข่ายประสาทเทียมมีอยู่หลายวิธีด้วยกัน แต่วิธีที่นิยมและแพร่หลายมากที่สุด คือ วิธีการเรียนรู้การเปลี่ยนแปลงของค่า w_{ji} โดยใช้วิธีที่เรียกว่าการแพร่กลับ (Back Propagation) การแพร่กลับพัฒนามาจากสมการพื้นฐานของการเรียนรู้แบบ McCulloch และ Pitts โดยการพิจารณาฟังก์ชันของความคลาดเคลื่อนระหว่างค่าเป้าหมายที่ต้องการ กับ ค่าของสัญญาณที่ส่งออกมาจากตัว neuron ในขอบข่ายของค่าจริงด้วย (ไม่ใช่แค่ 0 กับ 1 หรือ -1 กับ +1)

โครงข่ายประสาทเทียมที่มีการเรียนรู้โดยการแพร่กลับนี้จะประกอบไปด้วย กลุ่มของ เซลล์ประสาทต่างๆ จัดตัวกันเป็นชั้นๆ ดังนี้



รูปที่ 2.10 โครงข่ายประสาทเทียมที่พื้นฐาน

แต่ละขาของสัญญาณเข้าจะมีค่าความต้านทาน หรือ ค่าของสัญญาณออกของแต่ละเซลล์คำนวณได้จากสมการดังต่อไปนี้

$$f(\alpha) = \frac{1}{1 + e^{-(\alpha - \theta)}} \quad (\text{Sigmoid function}) \quad (2.2)$$

เมื่อ e คือค่า log ฐานธรรมชาติ

a คือ $\sum \omega_i X_i$

θ คือค่าคงที่

1. การกำหนดค่าความต้านทาน ω_i และ θ ของแต่ละเซลล์อย่างสุ่ม
2. ป้อนเวกเตอร์ $(X_1, X_2, \dots, X_n)$ และสัญญาณที่ต้องการให้ปรากฏที่ชั้นนอกสุด $(d_1, d_2, \dots, d_m)$
3. จากเวกเตอร์ $(X_1, X_2, \dots, X_n)$ คำนวณหาว่าสัญญาณที่ส่งออกจากเซลล์แต่ละเซลล์ที่ชั้นนอกสุด โดยไล่ไปที่ละชั้น จากชั้นที่ 1 การคำนวณโดยใช้สูตร $f(\alpha)$ ข้างต้น ให้

$(Y_1, Y_2, \dots, Y_n)$ เป็นค่าของสัญญาณที่ได้จากการคำนวณเวกเตอร์ $(d_1, d_2, \dots, d_m)$ จะแตกต่างจาก เวกเตอร์ $(Y_1, Y_2, \dots, Y_m)$

4.ปรับค่าของความต้านทานของแต่ละเซลล์ โดยเริ่มจากชั้นนอกสุดไล่ไปจนถึงชั้นที่ 1 ค่าความต้านทานใหม่ที่ปรับดังสมการต่อไปนี้

$$\omega_{ij}(t+1) = \omega_{ij}(t) + \eta \delta_j x'_j \quad (2.3)$$

เมื่อ ω_{ij} คือ ความต้านทานระหว่างเซลล์ประสาท i และ j ดังรูปด้านบน

η คือ อัตราการเรียนรู้ โดยที่ จะมีค่าไม่เกิน 1 (กำหนดเอง)

δ_j หาได้โดย

ก.ถ้าเซลล์ j เป็นเซลล์ชั้นนอกสุด ค่าของ j จะเท่ากับ

$$\delta_j = y_j(1-y_j)(d_j - y_j) \quad (2.4)$$

ข.ถ้าเซลล์ j เป็นเซลล์ชั้นที่ซ่อนอยู่ ค่าของ j จะเท่ากับ

$$\delta_j = x'_j(1-x'_j) \sum_k \delta_k \omega_{jk} \quad (2.5)$$

เมื่อ x'_j คือ ค่าสัญญาณขาเข้าของเซลล์ j

jk คือ จำนวนเซลล์ทั้งหมดบนชั้นที่อยู่ถัดจากเซลล์ j ขึ้นไป

5.ทำซ้ำข้อ2 ถึง ข้อ5 จนกว่าค่าความแตกต่าง (E) จะยอมรับได้ กำหนดดังนี้

$$E = \frac{1}{2} \sum_i (d_i - y_i)^2$$

หลังจากเราสอนโครงข่ายให้เรียนรู้แล้ว ต่อไปจะนำมันมาใช้งานโดยป้อนเวกเตอร์เข้าไป แล้วคำนวณหาค่าสัญญาณที่ออกมาจากแต่ละเซลล์โดยสมการซิกมอยด์(Sigmoid Function)

2.3 เทคนิคการแบ่งกลุ่ม (Clustering Technique)

จากแนวความคิดในเรื่องการจำแนกใบหน้าบุคคลนั้น จะต้องอาศัยปัจจัยหลายๆ อย่าง ประกอบกัน โดยที่ปัจจัยเหล่านั้นจะเกี่ยวข้องกันหรือไม่ก็ได้ จึงได้ทำการของ ปัจจัยเหล่านั้นเข้าด้วยกัน โดยในการจับกลุ่มนี้จะแทนด้วยปัจจัยที่จำเป็นด้วย Input node ของนิวรอลเน็ตเวิร์ค แล้วทำการจับกลุ่มด้วยวิธีการที่เรียกว่า K-Medoid

โดยวิธีการ K-Medoid นี้เป็นวิธีการแบ่งกลุ่มข้อมูลออกเป็นจำนวน K กลุ่ม โดยจะอาศัยข้อมูลบางตัวให้เป็นจุดศูนย์กลางของกลุ่มข้อมูล

จากสมการ

$$E = \sum_{j=1}^K \sum_{p \in C_j} |p - o_j| \quad (2.6)$$

เมื่อ	E	เป็นผลรวมของค่า absolute-error ของข้อมูลทั้งหมด
	C_j	เป็นกลุ่มที่ j
	K	เป็นจำนวนของกลุ่มทั้งหมด
	p	เป็นสมาชิกของแต่ละกลุ่มที่ไม่ใช่จุดอ้างอิง
	o_j	เป็นจุดอ้างอิงในกลุ่ม j

โดย Set ของ o_j ที่ทำให้มีค่า E ที่น้อยที่สุดจะทำให้ได้กลุ่มของข้อมูลที่เหมาะสม และสามารถหาค่า o_j ได้ดังนี้

กำหนดให้

K เป็นจำนวนของกลุ่มข้อมูลที่ต้องการแบ่ง

- 1) ทำการสุ่มค่าอ้างอิงขึ้นมาจากข้อมูลทั้งหมด สุ่มครั้งเป็นค่าอ้างอิงในแต่ละกลุ่ม
- 2) จับกลุ่มข้อมูลที่เหลือ โดยให้อยู่ในกลุ่มที่มีค่าอ้างอิงใกล้เคียงกันมากที่สุด
- 3) สุ่มค่าอ้างอิงขึ้นมาใหม่ 1 ค่า
- 4) คำนวณค่า E ค่าใหม่โดยแทนค่าอ้างอิงเดิมที่ละค่าด้วยค่าอ้างอิงใหม่ที่สุ่มขึ้นมา
- 5) ถ้าค่า E ที่คำนวณได้ใหม่มีค่าน้อยลง ให้ถือค่าอ้างอิงใหม่แทนค่าเดิมที่ถูกแทนแล้ว ทำให้มีค่า E ที่น้อยลง
- 6) ทำซ้ำตั้งแต่ 2 จนกว่าจะไม่มีเปลี่ยนแปลงค่าอ้างอิง